

Sequence Modeling Neural Networks - RNN, LSTM & GRU

S. R. Mahadeva Prasanna

Professor, Dept. of DSAI, IIT Dharwad

prasanna@iiitdwd.ac.in

October 7, 2024



Some Good Resources for NNDL

- Mithesh M. Khapra, "Deep Learning Part I and II" NPTEL Course.
- Ali Ghodsi, "Deep Learning", University of Waterloo Course.
- Duda and Hart, "Pattern Classification" Wiley, 2001.
- C. M. Bishop, "Neural Networks for Pattern Recognition", 1995.
- B. Yegnanarayana, "Artificial Neural Networks", PHI, 1999.
- C. M. Bishop, "PRML", Springer, 2006.
- Ian Goodfellow, et. al. , "Deep Learning", MIT Press, 2016.
- C. M. Bishop & H. Bishop, "Deep Learning", Springer, 2024
- Simon J. D. Prince, "Understanding Deep Learning", MIT Press, 2024



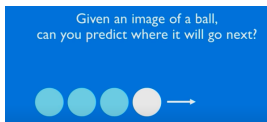
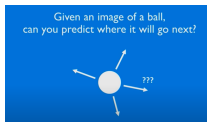
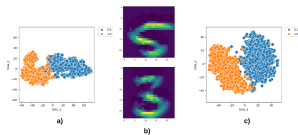
Organization

- Need for sequence modeling
- Recurrent Neural Network (RNN)
- Gated Recurrent Neural Network (GRNN)
- Long short term memory (LSTM) RNNs
- Bidirectional RNNs
- Time delay neural networks (TDNN)
- Short-term sequence information
- Long-term sequence information
- Attention mechanism
- Transformer network



Need for Sequence Modeling

- Class specific information is in distribution of data points
- Class specific information is in **sequential order**
- Hindi words Kalam vs Kamal

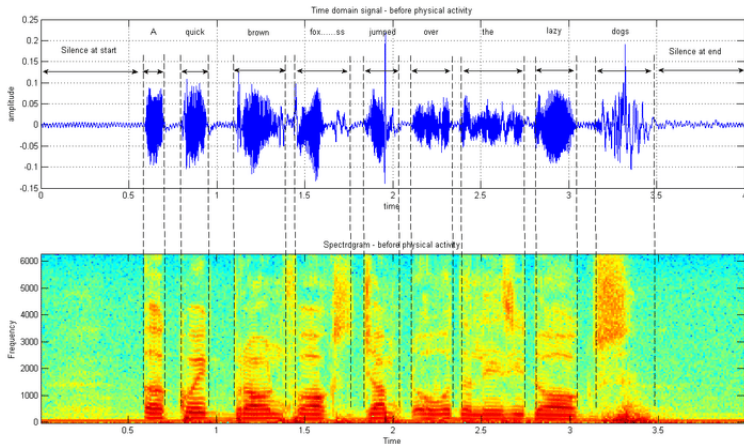


Need for Sequence Modeling (contd.)

- — name is —
- My name is —
- My city name is —
- My institute name is —
- Observe how answer depends on previous words in the sequence
- Answer only depends on what the sequence of past words are
- Observe how human brain is trained to exploit sequence knowledge
- How to mimic the same using deep learning models ?

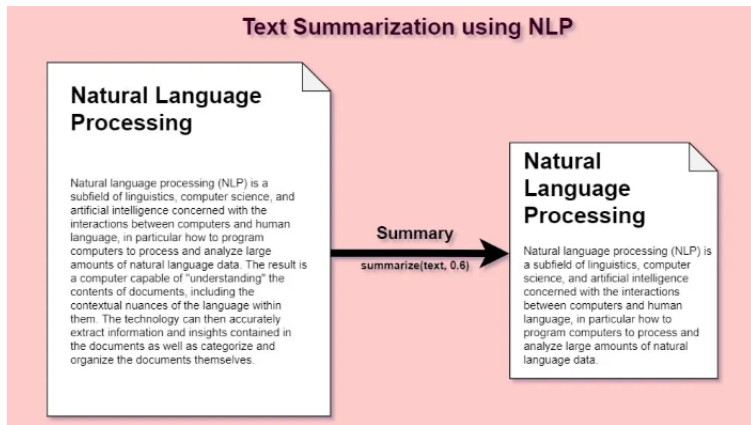


Sequence Modeling Motivation : Speech Recognition



Taken from public domain. Original authors acknowledged.

Sequence Modeling Motivation : Text Summarization



Taken from public domain. Original authors acknowledged.

Some Examples Involving Sequence Modeling

- **Speech Recognition** : Input (x) is speech signal and output (y) is sequence of recognized words (text).
- **Video Processing** : Input (x) is sequence of frames of video and output (y) is action depicted (text).
- **Music Processing** : Input (x) is a discrete entity (label) of genre and output (y) is music signal.
- **Speech to Speech Translation** : Input (x) is a speech signal in one language and output (y) is also speech signal in another language.
- **Sentiment Classification** : Input (x) is a text review of a product and output (y) is discrete rating.
- **Emotion Classification** : Input (x) is a speech / text description and output (y) is discrete emotion label.



How to do Sequence Modeling?

- Preserving the order in the given sequential data.
- Should be able to handle seqs of different lengths.
- No. of parameters independent of sequence length.
- Stochastic models like hidden Markov models (HMMs).
- Deep learning models that exploit and model sequence knowledge
- Recurrent neural networks (RNNs) and its variants like GRU, LSTM
- Gated Recurrent Units (GRUs) & Long Short Term Memory (LSTM)
- Time delay neural networks (TDNNs)



Significance of RNNs

- Consider a **name recognition task**.
- Consider **This fruit is Apple** and **Apple is fruit name**.
- Goal is to develop a model to read sentence and **extract fruit name**.
- **Apple** is relevant piece of information whichever position it is.
- Suppose a **MLP** is trained to process sentences of fixed length.
- **Separate MLP for each position** of word corresponding to name.
- **A single RNN shares the same weights** across several time steps.



Sequence Modeling : FFNN Limitation

Idea #1: Use a Fixed Window

"This morning I took my cat for a walk."
given these predict the
two words next word

- Use of fixed window has limited history

Problem #1: Can't Model Long-Term Dependencies

"France is where I grew up, but I now live in Boston. I speak fluent ____."

- Fixed window cannot model longterm dependency
- Longterm dependency can be of variable length

Ava Soleimany, "Deep Sequence Modeling " MIT 6.S191 (2020): Recurrent Neural Networks".



Can I do Sequence Modeling by FFNN ?

- Randomizing and applying feature vectors one after other loses sequence knowledge completely.
- Parallel in and parallel out configuration may be alternative. I/p and o/p vectors can be of very large dimension and leads to memorizing.
- Further parallel way of applying will not give any opportunity to learn sequence knowledge as there is no sharing of knowledge.
- It is not possible as the input and output vector lengths may be different for different examples.

Limitation of FFNN for Sequence Modeling

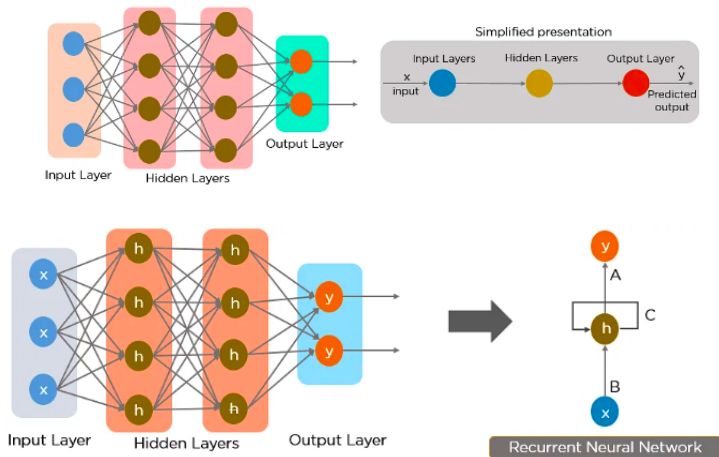
- Need of fixed window is not possible.
- Cannot model variable length longterm dependency
- Cannot model word order.
- Cannot share parameters.

Looking out for New Architecture

- Can handle variable length window.
- Can model variable length longterm dependency
- Can preserve word order.
- Can share parameters.



FFNN to RNN



Taken from public domain. Original authors acknowledged.

About Recurrent Neural Network (RNN)

- Rumelhart et al., are the proposers of RNNs in 1986.
- RNNs are a family of neural networks for **processing sequential data**.
- **CNN** specialized for **processing a grid** of values **$\mathbf{X}$** such as an image.
- Alternatively, **RNN** is a NN that is specialized for **processing a sequence** of values $\mathbf{x}(1), \dots, \mathbf{x}(\tau)$.
- **Speech Recognition** : Input is seq. of feature vectors (x), say $x(1), x(2), \dots$ and output is seq. of text words (y), say, $y(1), y(2), \dots y(5)$
- **Named Entity Recognition** : Input is seq. of text words (x), say $x(1), x(2), \dots x(10)$ and output is seq. of binary digits (y), say, $y(1), y(2), \dots y(10)$, 1 represent named entity and 0 other word.

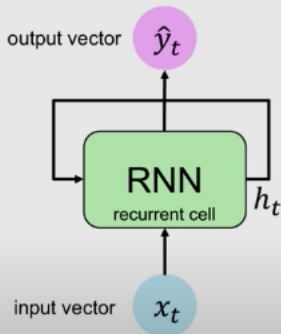


About Recurrent Neural Network (RNN)

- Key idea in moving from MLP to RNN is **sharing parameters across different parts of a model**.
- Sharing parameters across different hidden layers as opposed to independent wts in case of MLP
- Sharing parameters helps to **generalizing to variable sequence lengths, share statistical strength and also across different positions in time**.
- Sharing is particularly important when a specific piece of **information can occur at multiple positions within the sequence**.
- Sharing helps in a significantly **optimized structure** compared to MLP.
- Some sort of recurrence (happening again and again) in the network

RNN Recurrence Relation

Recurrent Neural Network (RNN)



Apply a **recurrence relation** at every time step to process a sequence:

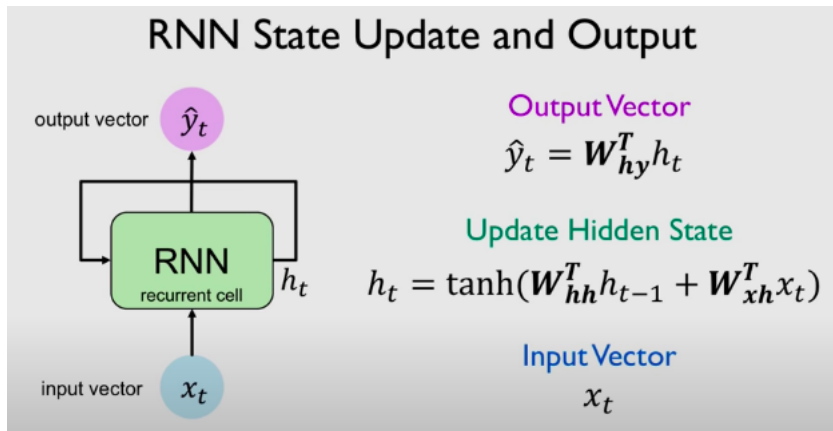
$$h_t = f_W(h_{t-1}, x_t)$$

cell state function parameterized by W old state input vector at time step t

Note: the same function and set of parameters are used at every time step

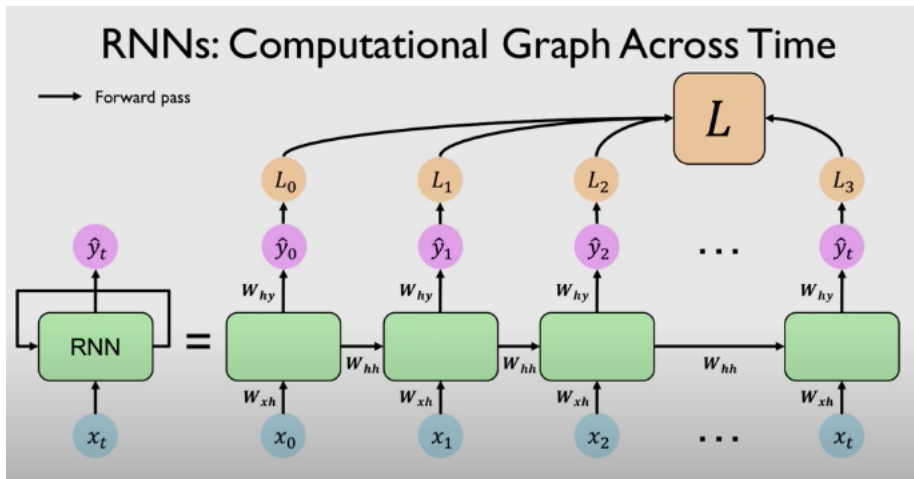
Ava Soleimany, "Deep Sequence Modeling "MIT 6.S191 (2020): Recurrent Neural Networks".

RNN Weights (Parameters) Update



Ava Soleimany, "Deep Sequence Modeling " MIT 6.S191 (2020): Recurrent Neural Networks".

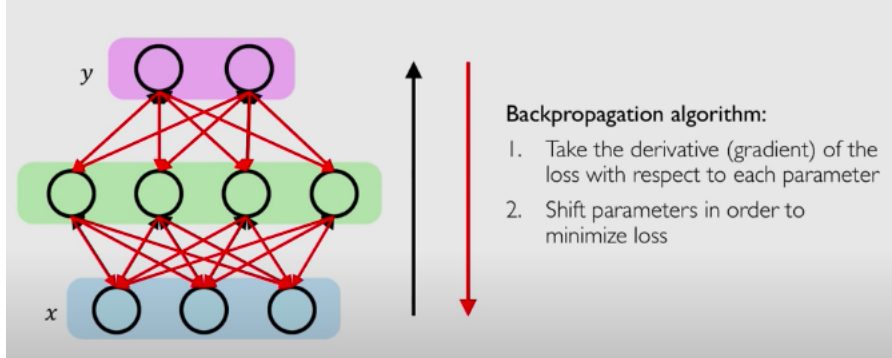
RNN Loss Computation



Ava Soleimany, "Deep Sequence Modeling " MIT 6.S191 (2020): Recurrent Neural Networks".

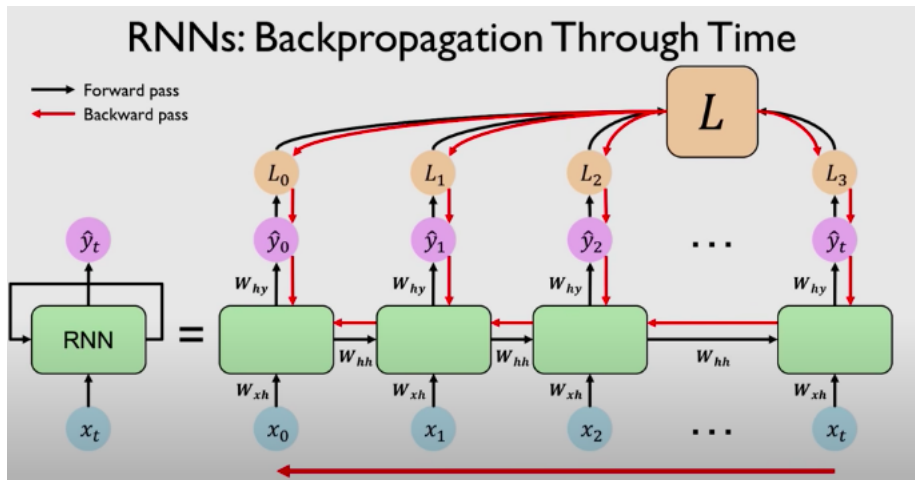
Back Propagation Algorithm

Recall: Backpropagation in Feed Forward Models



Ava Soleimany, "Deep Sequence Modeling " MIT 6.S191 (2020): Recurrent Neural Networks".

Back Propagation Through Time Algorithm



Ava Soleimany, "Deep Sequence Modeling " MIT 6.S191 (2020): Recurrent Neural Networks".

BPTT Approach - Chain Rule of Differentiation

- Many to one RNN architecture is taken as reference
- w_x, w_h, w_o are weights associated with input, hidden and output
- Three sequence values, x_1, x_2, x_3 are i/p sequence values
- Hidden state values are h_0, h_1, h_2, h_3
- $h_1 = f(w_x x_1 + w_h h_0)$
- $h_2 = f(w_x x_2 + w_h h_1)$
- $h_3 = f(w_x x_3 + w_h h_2)$
- Output $\hat{y}_3 = \sigma(w_o h_3)$
- $E = y - \hat{y}$



BPTT Approach - Chain Rule of Differentiation

- Output $\hat{y}_3 = \sigma(w_o h_3)$
- $E = y - \hat{y}$ depends on w_x, w_h, w_o
- Minimize loss
- Gradient descent
- $w_o = w_o - \eta \frac{\partial E}{\partial w_o}$
- $w_h = w_h - \eta \frac{\partial E}{\partial w_h}$
- $w_x = w_x - \eta \frac{\partial E}{\partial w_x}$



$$\frac{\partial E}{\partial w_o}, \quad \frac{\partial E}{\partial w_h}, \quad \frac{\partial E}{\partial w_x}$$

- $\frac{\partial E}{\partial w_o} = \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial w_o}$
- $\frac{\partial E}{\partial w_h}$, w_h depends on h_3, h_2, h_1
- $\frac{\partial E}{\partial w_h} = \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_3} \frac{\partial h_3}{\partial w_h} + \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial w_h} + \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w_h}$
- $\frac{\partial E}{\partial w_h} = \sum_{j=1}^3 \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_j} \frac{\partial h_j}{\partial w_h}$
- $\frac{\partial E}{\partial w_x} = \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_3} \frac{\partial h_3}{\partial w_x} + \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial w_x} + \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_3} \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial w_x}$
- $\frac{\partial E}{\partial w_x} = \sum_{j=1}^3 \frac{\partial E}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h_j} \frac{\partial h_j}{\partial w_x}$

Comparison on performance on classification task between FFNN & RNN

Table: A comparison of sentiment classification of reviews using IMDB dataset by varying the length of input sequence between Feed Forward Neural Network and RNN

Sl. No	Seq. length	FFNN		RNN	
		Time (in sec)	Accuracy (in %)	Time (in sec)	Accuracy (in %)
1	50	82.43	50.52	563.20	81.03
2	100	82.46	50.38	1043.04	82.30
3	200	54.33	50.84	2024.88	53.04
4	300	82.75	50.41	2843.07	51.79

Basic Assumptions in RNNs

- RNNs operate on a sequence that contains vectors $\mathbf{x}(t)$ with the time step index t ranging from 1 to τ .
- In practice, RNNs operate on minibatches of such sequences, with a different sequence length τ .
- Time step index refer to both passage of time and position in the sequence.
- Even when applied to data involving time, the network may have connections that go backwards in time.



Working Principle of RNN

- To begin, takes input $\mathbf{x}(1)$ and activation value $\mathbf{h}(0)$ and predicts output $\hat{\mathbf{y}}(1)$ by passing through traditional NN architecture pipeline. $\mathbf{h}(0)$ is zero or random.
- Takes input $\mathbf{x}(2)$ and activation value $\mathbf{h}(1)$ and predicts output $\hat{\mathbf{y}}(2)$ by passing through traditional NN architecture pipeline.
- Continues till final step taking input $\mathbf{x}(\tau)$ and activation value $\mathbf{h}(\tau - 1)$ and predicts output $\hat{\mathbf{y}}(\tau)$ by passing through traditional NN architecture pipeline.
- The RNN scans through the data from left to right.
- Same parameters (w_{hx} , w_{hh} , w_{yh}) are shared across input, hidden and output connections.



Working Principle of RNN

- While predicting $\hat{\mathbf{y}}(3)$ information not only from $\mathbf{x}(3)$, but also from $\mathbf{x}(2)$ and $\mathbf{x}(1)$ are used through w_{hh} .
- $\mathbf{h}(1) = g(w_{hh}\mathbf{h}(0) + w_{hx}\mathbf{x}(1) + b_h)$ and $\hat{\mathbf{y}}(1) = g(w_{yh}\mathbf{h}(1) + b_y)$.
- $\mathbf{h}(t) = g(w_{hh}\mathbf{h}(t-1) + w_{hx}\mathbf{x}(t) + b_h)$ and $\hat{\mathbf{y}}(t) = g(w_{yh}\mathbf{h}(t) + b_y)$.
- $\mathbf{h}(t) = g(w_h[\mathbf{h}(t-1), \mathbf{x}(t)] + b_h)$ and $\hat{\mathbf{y}}(t) = g(w_y\mathbf{h}(t) + b_y)$.
- $w_h = [w_{hh}; w_{hx}]$

Forward propagation and Backpropagation Through RNNs

- **Forward Propagation:** $\mathbf{h}(t) = g(w_h[\mathbf{h}(t-1), \mathbf{x}(t)] + b_h)$ and $\hat{\mathbf{y}}(t) = g(w_y \mathbf{h}(t) + b_y)$, where $w_h = [w_{hh}; w_{hx}]$
- **Backpropagation:** Cost function $J(t)(\hat{\mathbf{y}}(t), y(t)) = -y(t)\log\hat{\mathbf{y}}(t) - (1 - y(t))\log(1 - \hat{\mathbf{y}}(t))$.
- $J = \sum_{t=1}^{\tau} J(t) = \sum_{t=1}^{\tau} J(t)(\hat{\mathbf{y}}(t), y(t))$
- Conventional backpropagation for each $J(t)$ from output to input node at t .
- **Backpropagation through time** is using J and moving across time.

Vanishing Gradient Problem in RNNs

- Both conventional backpropagation and backpropagation through time are present in RNNs.
- Since the networks are deep, naturally leads to vanishing gradient problem.
- Leads to what is called **local effect / influence**. Any output is influenced by input and activation values in neighbourhood.
- Thus vanishing gradient is a bigger problem in case of RNNs.
- If we do not address, then it is very difficult to model longterm dependencies which is the basic purpose of RNNs !



Short vs Long Term Dependency

- Language model trying to predict next word based on previous ones
- Short Term :
 - "the clouds are in the sky"
 - To predict "sky" do not need further information
 - Treated as short term dependency
 - Can be handled by RNN



Short vs Long Term Dependency

- Long Term :

- "I am from a South Indian family, specifically Karnataka, but settled in Delhi. I had spent most of my childhood and done schooling as well as college education in Delhi. Apart from Kannada, I can speak fluent Hindi also"
- Short term dependency suggests next word is probably name of a language.
- However, to narrow down which language, need context of Delhi, from further back (Long Term)
- Gap between relevant information and point where it is needed to become very large
- Unfortunately, as that gap grows, RNNs become unable to learn to connect the information.



Limitations of RNNs

- When the length of sequence is large, RNNs show degradation in performance due to vanishing or exploding gradient issue
- RNN will not be able to model long term context (LTC) information
- RNN can only model short term context (STC) information

Limitation of RNN

- Need to modify RNN architecture to take care of both STC and LTC
- Two feedback in RNN cell, one for STC and another for LTC
- And mechanism (gates) for communication among STC and LTC feedback links, when needed
- Modify basic recurrence cell of RNN to accommodate the same
- Results in Long Short Term Memory (LSTM) neural network

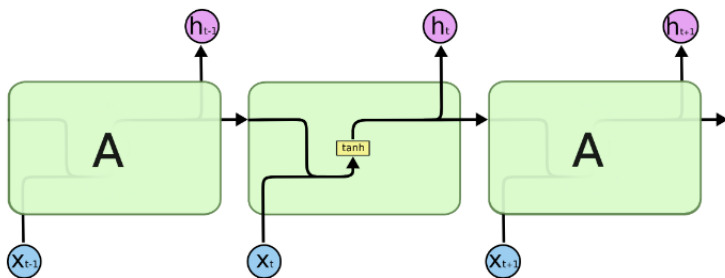


Long Short Term Memory (LSTM) Neural Network

- LSTM is a special kind of RNN
- Performs much better than RNN in many tasks
- Most exciting results of RNN are through LSTM variant.
- Will have modified ver of recurrence cell to store information about both short and long term dependencies
- LSTMs are explicitly designed to avoid the long-term dependency problem.

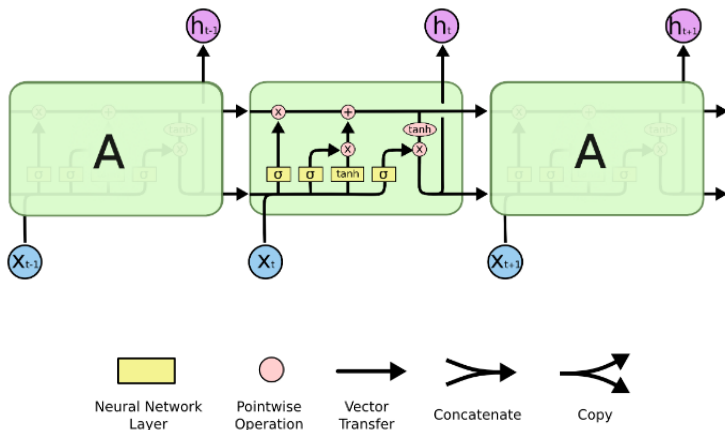


Recurrence Cell in RNN



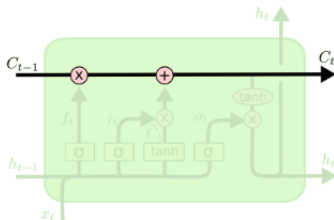
Acknowledgement - Understanding LSTM Networks, Colah's Blog.

Recurrence Cell in LSTM



Acknowledgement - Understanding LSTM Networks, Colah's Blog.

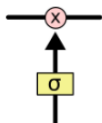
Cell State in LSTM



- **Cell state**, horizontal line running through top of diagram
- LSTM can remove or add information to cell state, regulated by structures called **gates**.

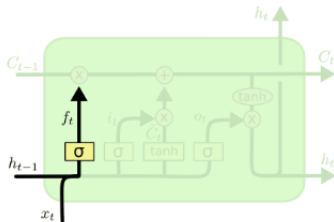
Acknowledgement - Understanding LSTM Networks, Colah's Blog.

Gates in LSTM



- **Gates** way to optionally let information through.
- A sigmoid neural net layer and a pointwise multiplication.
- Sigmoid outputs between zero and one, describing how much of each component should be let through.
- Zero “let nothing through,” one “let everything through!”
- LSTM has **three gates**, to protect and control cell state.

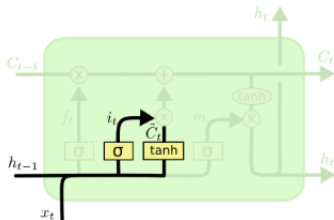
Forget Gate in LSTM



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

- Decides what information to throw away from the cell state.
- Decision made by a sigmoid layer called “forget gate layer”
- It looks at h_{t-1} and x_t and outputs between 0 and 1 for each number in the cell state C_{t-1}
- $1 \implies$ “completely keep this”, $0 \implies$ “completely get rid of this.”

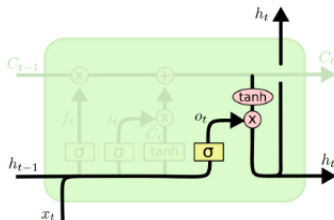
Input Gate in LSTM



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

- Decides what new information to store in the cell state.
- A sigmoid layer “input gate layer” decides which values to update
- A tanh layer creates a vector of new values, $\tilde{C}_t$, to be added
- Combine these two to create an update to the state
- Update old cell state C_{t-1} to new cell state C_t

Output Gate in LSTM



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

- Decides what to output as h_t .
- Based on cell state, but a filtered version
- Sigmoid layer decides what parts of the cell state to output
- Put cell state through $\tanh$ (-1 and 1) and multiply by output of sigmoid, so that only output the parts we decided to.

Classification task using IMDB Movie Review dataset

Internet Movie Database (IMDB) dataset:

- Large collection of movie reviews from IMDB website.
- Benchmark dataset for various NLP applications such as sentiment analysis, text classification, and language modeling.
- Contains about 50,000 movie reviews labelled as positive or negative based on its sentiment.
- Example reviews:
 - *"This movie was absolutely fantastic! The storyline was gripping, the acting was superb, and ... "* **(Positive)**
 - *"This movie was just plain bad"* **(Negative)**



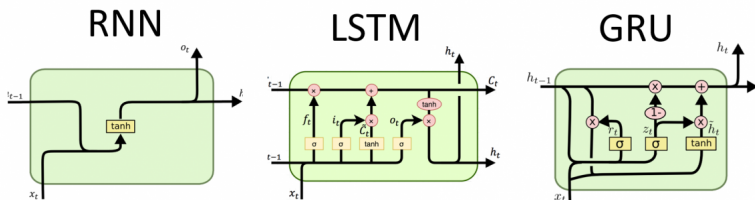
Performance comparison between RNN & LSTM

Table: Comparison of performances between RNN & LSTM on Sentiment Classification task using the IMDB Movie Review dataset

Sl. No	Seq. length	RNN		LSTM	
		Time (in sec)	Accuracy (in %)	Time (in sec)	Accuracy (in %)
1	50	563.20	81.03	153.44	81.69
2	100	1043.04	82.30	203.79	85.01
3	200	2024.88	53.04	263.67	87.16
4	300	2843.07	51.79	324.07	78.28

Gated Recurrent Unit (GRU) Network

- GRU is a type of RNN
- Can process sequential data such as time series
- Difference between a GRU and other RNN architectures, such as LSTM network, is how it handles information flow through time.
- 1 state & 2 gates (2σ) vs 2 states & 3 gates (3σ) of LSTM
- Single state carries both STC and LTC information



GRU vs LSTM

	GRU	LSTM
Structure	Simpler structure with two gates (update and reset gate)	More complex structure with three gates (input, forget, and output gate)
Parameters	Fewer parameters (3 weight matrices)	More parameters (4 weight matrices)
Training	Faster to train	Slow to train

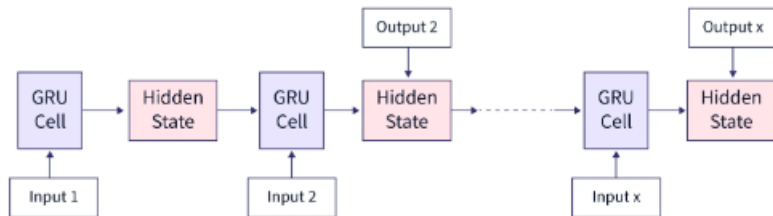
Scaler Topics, <https://www.scaler.com/topics/deep-learning/gru-network/>



GRU vs LSTM

Space Complexity	In most cases, GRU tend to use fewer memory resources due to its simpler structure and fewer parameters, thus better suited for large datasets or sequences.	LSTM has a more complex structure and a larger number of parameters, thus might require more memory resources and could be less effective for large datasets or sequences.
Performance	Generally performed similarly to LSTM on many tasks, but in some cases, GRU has been shown to outperform LSTM and vice versa. It's better to try both and see which works better for your dataset and task.	LSTM generally performs well on many tasks but is more computationally expensive and requires more memory resources. LSTM has advantages over GRU in natural language understanding and machine translation tasks.

GRU Architecture



Scaler Topics, <https://www.scaler.com/topics/deep-learning/gru-network/>

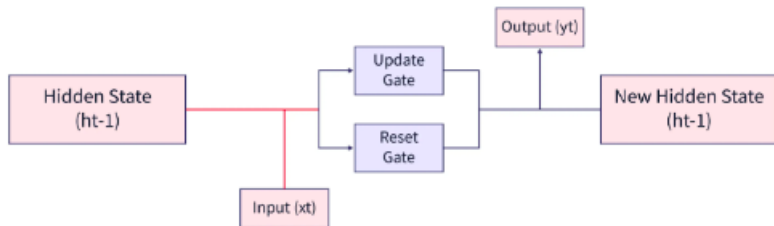
GRU Architecture

- GRU comprises three main components: an update gate, a reset gate, and a candidate hidden state
- Cell takes decision on how much to be considered from the past state and how much should be forgotten
- GRU cell has two gates: Reset Gate & Update Gate.

Scaler Topics, <https://www.scaler.com/topics/deep-learning/gru-network/>



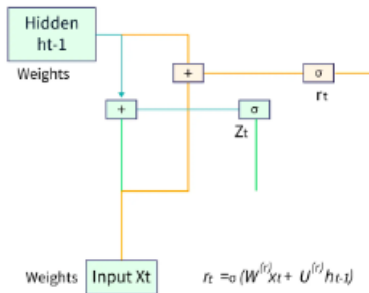
GRU Cell



Scaler Topics, <https://www.scaler.com/topics/deep-learning/gru-network/>

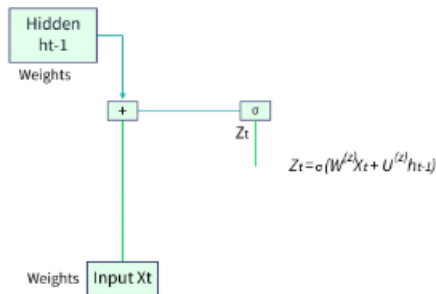
Reset Gate

- Identifies unnecessary information and decides what information to be laid off from the GRU network
- What information to delete at the specific timestamp
- Same as forget gate in LSTM
- Output of Reset Gate : $r_t = \sigma(W^{(r)}x_t + U^{(r)}h_{(t-1)})$



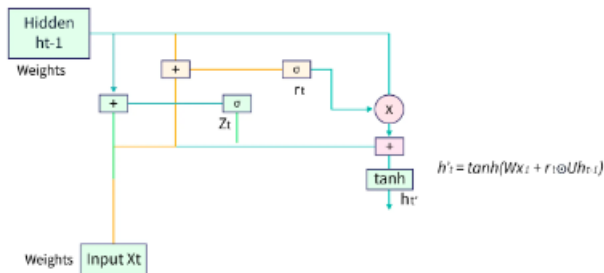
Update Gate

- Finds what current GRU cell need to be passed onto the next GRU cell
- Keeping track of the most important information
- Same as output gate in LSTM
- Output of Reset Gate : $z_t = \sigma(W^{(z)}x_t + U^{(z)}h_{(t-1)})$



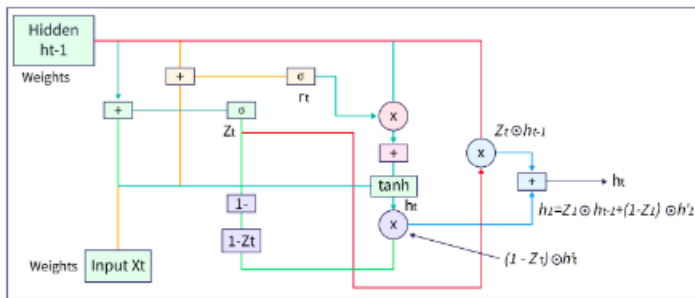
Hidden State - Candidate

- Hidden state is a vector that captures information from past time steps relevant to current time step
- Diff dim of hidden state will carry weighted values of STC and LTC
- **Candidate Hidden State** : $h'_t = \tanh(Wx_t + r_t h_{(t-1)})$



Hidden State - Final

- **Hidden State** : $h_t = z_t h_{(t-1)} + (1 - z_t) h'_t$



Scaler Topics, <https://www.scaler.com/topics/deep-learning/gru-network/>



Bi-Directional Sequence Models

- Sequence modeling involves deriving knowledge from the sequence for prediction or classification
- Conventional RNN, LSTM and GRU are designed to look only the sequence from past
- Many applications where future part of sequence needs to be looked into or both
- "Aswathhama hathaha (and then he murmured) iti Narova Kunjarova"
- "Aswathhama is dead (and then he murmured). I don't know if its a man or an Elephant."

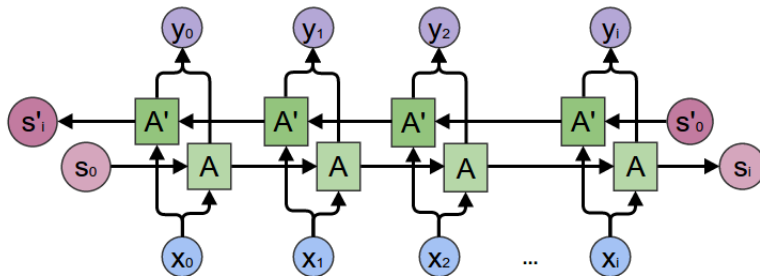


Bi-Directional Sequence Models

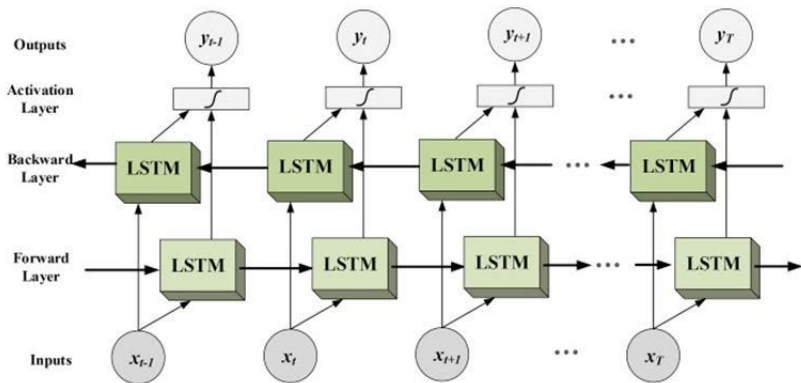
- Named entity recognition (NER) - Ashoka is Person or place ?
- Ashoka is a great warrior
- Ashoka is a good hotel
- I went to Cauvery and took a bath
- Cauvery is person or river ?
- Need for looking into sequence knowledge both from past and future
- Implemented using Bi-directional sequence models



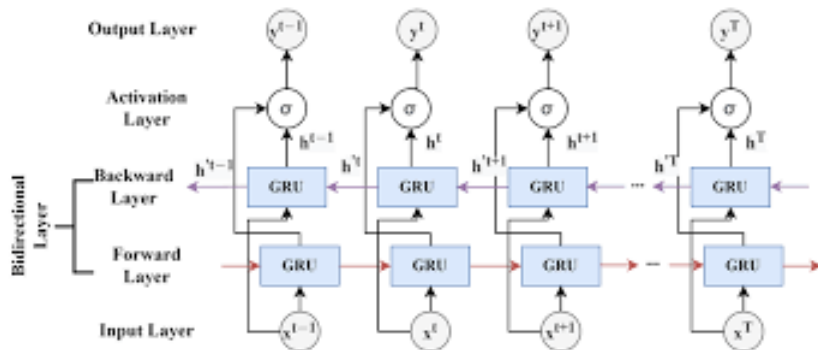
Bi-Directional RNN



Bi-Directional LSTM



Bi-Directional GRU



Bidirectional RNNs

- **Forward Direction:** $\mathbf{a}(t) = g(w_a[\mathbf{a}(t-1), \mathbf{x}(t)] + b_a)$
- **Backward Direction:** $\mathbf{b}(t) = g(w_b[\mathbf{b}(t+1), \mathbf{x}(t)] + b_a)$
- **Output:** $\hat{\mathbf{y}}(t) = g(w_y[\mathbf{a}(t), \mathbf{b}(t)] + b_y)$

Deep Architecture based Sequence Modeling

- Significance of deep learning and deep architecture has been proven beyond doubt in DNN and CNNs
- Same thing will be true for sequence models also
- Accordingly Deep RNN, Deep LSTM and Deep GRU architectures
- Representation learning is one area
- Complex machine learning task is another area

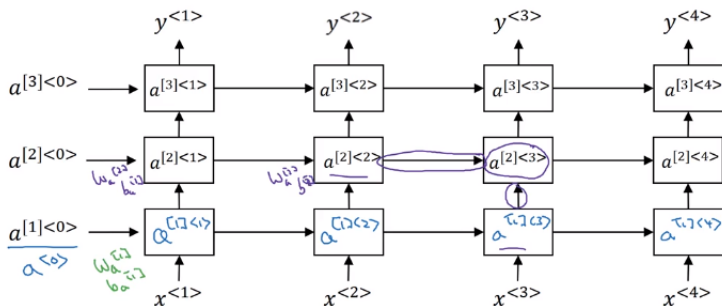


Deep vs Shallow Sequence Models

- Typically 1 or 2 to many hidden layers
- In shallow architecture hidden state information is passed onto to produce output
- In deep architecture one layer hidden state information is passed to next layer input information
- Process continues till required representation or classification result is achieved
- At the last hidden layer, the final output will be computed



Deep RNN



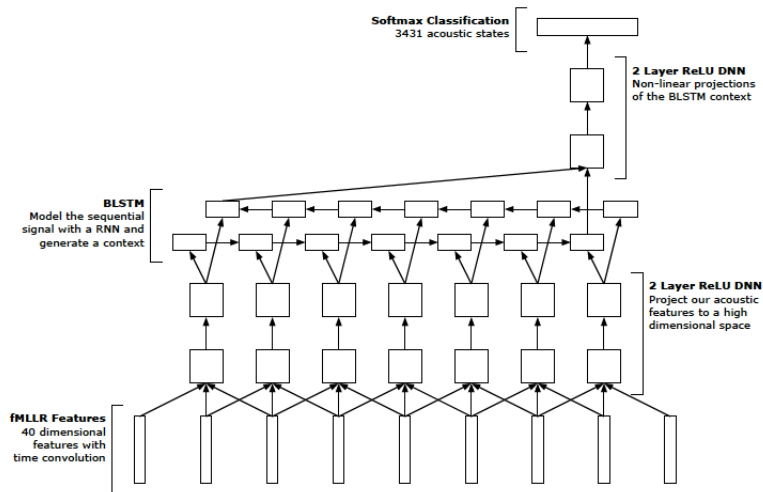
$$a^{[2] < 3 >} = g(w_a^{[2]} [a^{[1] < 2 >}, a^{[1] < 3 >}] + b_a^{[2]})$$

Deep RNNs

- Several layers of RNNs at each instant of time to get deeper ver.
- **RNNs:** $\mathbf{a}_1(t) = g(w_{a_1}[\mathbf{a}_1(t-1), \mathbf{x}(t)] + b_{a_1})$
- **RNNs:** $\mathbf{a}_2(t) = g(w_{a_2}[\mathbf{a}_2(t-1), \mathbf{a}_1(t)] + b_{a_2})$
- **RNNs:** $\mathbf{a}_l(t) = g(w_{a_l}[\mathbf{a}_l(t-1), \mathbf{a}_{(l-1)}(t)] + b_{a_l})$
- **Output:** $\hat{\mathbf{y}}(t) = g(w_y \mathbf{a}_l(t) + b_y)$
- Similar deep ver can be obtained for GRU, LSTM and BRNN



Deep Recurrent Neural Networks for Acoustic Modelling



William Chan & Ian Lane, "Deep recurrent neural network for acoustic modelling" <https://doi.org/10.48550/arXiv.1504.01482>



Performance of Deep RNN based Acoustic Modelling

Model	dev93 WER	eval92 WER
GMM Kaldi tri4b	9.39	5.39
DNN Kaldi s5	6.68	3.81
DNN ReLU	6.84	3.79

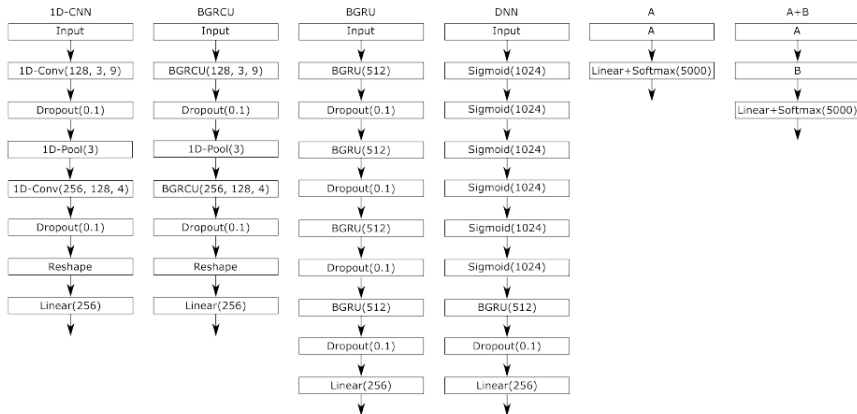
Cell Size	Layers	dev93 WER	eval92 WER
128	1	8.19	5.19
256	1	7.94	4.66
512	1	7.43	4.36
768	1	7.36	4.16
1024	1	7.23	4.06
256	2	7.54	4.36
512	2	7.40	4.25

Model	dev93 WER	eval92 WER
DNN-BLSTM	7.40	3.92
BLSTM-DNN	6.90	3.84
DNN-BLSTM-DNN	7.19	3.76
TC-DNN-BLSTM-DNN	6.58	3.47

William Chan & Ian Lane, "Deep recurrent neural network for acoustic modelling" <https://doi.org/10.48550/arXiv.1504.01482>



Deep GRU for Acoustic Modelling



Markus Nussbaum-Thom, Jia Cui, Bhuvana Ramabhadran, Vaibhava Goel, "Acoustic Modeling using Bidirectional Gated Recurrent Convolutional Units", Interspeech 2016



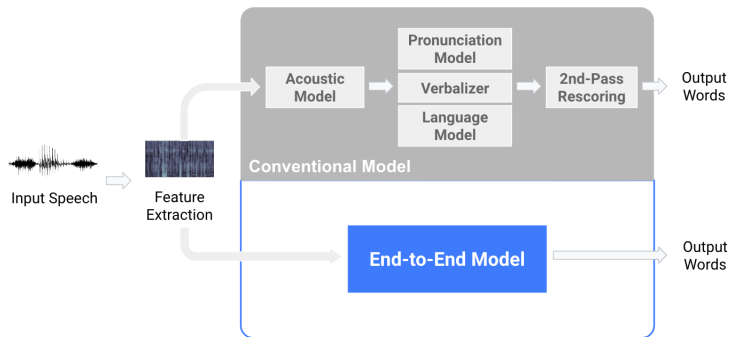
Performance of Deep GRU based Acoustic Modelling

	WER[%]		
model	model size	DEV04F	RT04
DNN	10 M	16.2	15.8
2D-CNN+DNN	66 M	15.8	15.0
BGRU	24 M	14.2	13.6
1D-CNN	4 M	26.8	25.4
1D-CNN+DNN	14 M	24.3	22.1
1D-CNN+BGRU	32 M	14.1	13.6
BGRCU	11 M	15.4	14.7
BGRCU+DNN	14 M	15.6	14.5
BGRCU+BGRU	37 M	13.8	13.2

Markus Nussbaum-Thom, Jia Cui, Bhuvana Ramabhadran, Vaibhava Goel, "Acoustic Modeling using Bidirectional Gated Recurrent Convolutional Units", Interspeech 2016

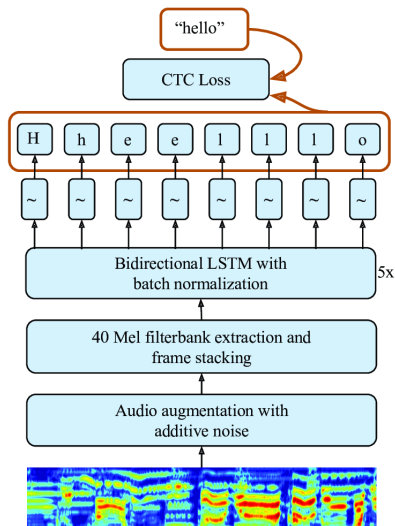


Block Diagram of End to End (E2E) ASR System



Tara Sainath "End-to-End Speech Recognition: The Journey from Research to Production", Invited Talk, EEML Summer School, July 2022. (also given at SANE - October 2022; Nvidia - November 2022).

Grapheme based E2E ASR



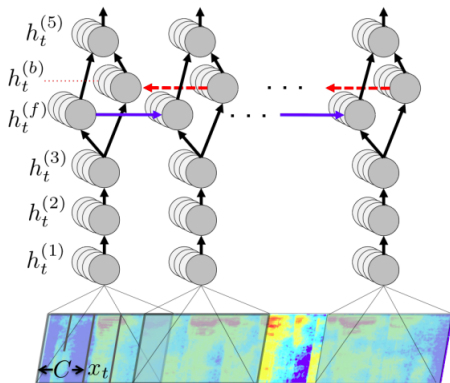
End to End ASR

- Feature representation from raw waveform/filter bank features: local context.
- Acoustic modeling: Feature to the character.
- Alignment with CTC: Global context (character to word)
- End-to-End Models:
 - 1 Deep Speech (Google)
 - 2 W2V (Facebook)
 - 3 Whisper (OpenAI)



Deep Speech ASR

- DNN-HMM, DL only for acoustic modeling
- End-to-end ASR system by Baidu's Silicon Valley AI Lab (SVAIL) in 2014
- DNN architecture with recurrent layers and spectrogram as input
- Trained using a supervised learning approach with CTC



Awani Y et al., "Deep Speech: Scaling up end-to-end speech recognition", arXiv, 2014.

Databases for Deep Speech ASR

Dataset	Type	Hours	Speakers
WSJ	read	80	280
Switchboard	conversational	300	4000
Fisher	conversational	2000	23000
Baidu	read	5000	9600

Awani Y et al., "Deep Speech: Scaling up end-to-end speech recognition", arXiv, 2014.

Performance on various datasets

Table: Published Word error rates (%WER) on Switchboard dataset splits. The columns labeled Switchboard as “SWB” and CallHome as “CH” are, respectively, the easy and hard subsets of Hub5’00.

Model	SWB	CH	Full
Vesely et al. (GMM-HMM BMMI) [44]	18.6	33.0	25.8
Vesely et al. (DNN-HMM sMBR) [44]	12.6	24.1	18.4
Maas et al. (DNN-HMM SWB) [28]	14.6	26.3	20.5
Maas et al. (DNN-HMM FSH) [28]	16.0	23.7	19.9
Seide et al. (CD-DNN) [39]	16.1	n/a	n/a
Kingsbury et al. (DNN-HMM sMBR HF) [22]	13.3	n/a	n/a
Sainath et al. (CNN-HMM) [36]	11.5	n/a	n/a
Soltau et al. (MLP/CNN+I-Vector) [40]	10.4	n/a	n/a
Deep Speech SWB	20.0	31.8	25.9
Deep Speech SWB + FSH	12.6	19.3	16.0

Awni Y et al., "Deep Speech: Scaling up end-to-end speech recognition", arXiv, 2014.



Performance on various datasets

System	Clean (94)	Noisy (82)	Combined (176)
Apple Dictation	14.24	43.76	26.73
Bing Speech	11.73	36.12	22.05
Google API	6.64	30.47	16.72
wit.ai	7.94	35.06	19.41
Deep Speech	6.56	19.06	11.85

Awni Y et al., "Deep Speech: Scaling up end-to-end speech recognition", arXiv, 2014.

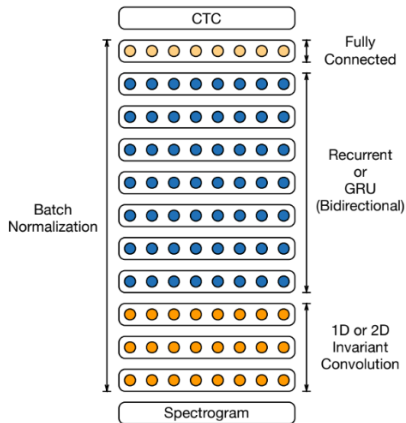
Deep Speech 2 ASR

- Deep Speech 2: End-to-end ASR by Baidu's in 2016
- Improved ver of Deep Speech
- English or Mandarin Chinese speech
- Different noise, accent, dialect and languages
- System is comparable to humans on standard datasets
- Addresses range of speech recognition contexts handled by humans.
- Significant reduction in WER for English and Mandarin with high accuracy.



Architecture of Deep Speech 2 ASR

- 3 layers of 2D-invariant convolution
- 7 layers of bidirectional simple recurrence
- 100M parameters



Dario Amodei et al., "Deep Speech 2: End-to-End Speech Recognition in English and Mandarin", International Conference on Machine Learning, PMLR, 2016.

Deep Speech 2 ASR Datasets

- English speech system trained on 11,940 hours of speech
- Mandarin system is trained on 9,400 hours.
- Data synthesis to further augment data during training.

Table: English Datasets.

Dataset	Speech Type	Hours
WSJ	read	80
Switchboard	conversational	300
Fisher	conversational	2000
LibriSpeech	read	960
Baidu	read	5000
Baidu	mixed	3600
Total		11940

Performance on various depths & batch normalization

Table: Comparison of WER for various depths of RNN and batch normalization.

Architecture	Hidden Units	Train		Dev	
		Baseline	BatchNorm	Baseline	BatchNorm
1 RNN, 5 total	2400	10.55	11.99	13.55	14.40
3 RNN, 5 total	1880	9.55	8.29	11.61	10.56
5 RNN, 7 total	1510	8.59	7.61	10.77	9.78
7 RNN, 9 total	1280	8.76	7.68	10.83	9.52

Dario Amodei et al., "Deep Speech 2: End-to-End Speech Recognition in English and Mandarin", International Conference on Machine Learning, PMLR, 2016.



Performance on simple RNN vs GRU

Table: Comparison of WER for RNN vs GRU .

Architecture	Simple RNN	GRU
5 layers, 1 Recurrent	14.40	10.53
5 layers, 3 Recurrent	10.56	8.00
7 layers, 5 Recurrent	9.78	7.79
9 layers, 7 Recurrent	9.52	8.19

Dario Amodei et al., "Deep Speech 2: End-to-End Speech Recognition in English and Mandarin", International Conference on Machine Learning, PMLR, 2016.

Effect of Language Model

Table: Effect of Language Model.

Language	Architecture	Dev no LM	Dev LM
English	5-layer, 1 RNN	27.79	14.39
English	9-layer, 7 RNN	14.93	9.52
Mandarin	5-layer, 1 RNN	9.80	7.13
Mandarin	9-layer, 7 RNN	7.55	5.81

Dario Amodei et al., "Deep Speech 2: End-to-End Speech Recognition in English and Mandarin", International Conference on Machine Learning, PMLR, 2016.

Performance on various datasets

Table: Comparison of WER for two speech systems and human level performance on read speech.

Read Speech			
Test set	DS1	DS2	Human
WSJ eval'92	4.94	3.60	5.03
WSJ eval'93	6.94	4.98	8.08
LibriSpeech test-clean	7.89	5.33	5.83
LibriSpeech test-other	21.74	13.25	12.69

Dario Amodei et al., "Deep Speech 2: End-to-End Speech Recognition in English and Mandarin", International Conference on Machine Learning, PMLR, 2016.



Thank You

